Name: Dhruv Jain

Roll No: 2301045

Batch: T12

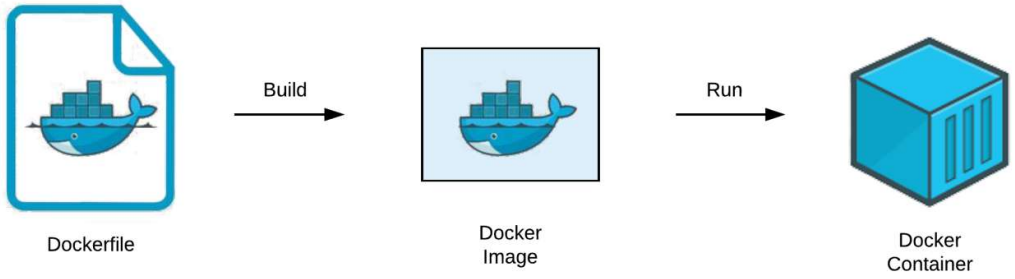
Experiment 10

Aim: To learn Dockerfile instructions, build an image for a sample web application using DOCKERFILE.

Theory:

What is a Dockerfile?

A Dockerfile is essentially a text file (literally written as "Dockerfile" with no extension) that contains instructions for creating a Docker image. These instructions are written in the format: DIRECTIVE argument. Although the directive is case-insensitive, it is recommended to write all directives in uppercase to differentiate them from arguments. A Dockerfile usually consists of multiple lines of instructions that are executed sequentially by the Docker engine during the image-building process.



Core Dockerfile Directives

- FROM (Specify the base image): The very first instruction in a Dockerfile must be the FROM directive. The base image is the foundation from which all other layers in your Docker image will be built, and it is downloaded from Docker Hub, Docker's official image registry.
- WORKDIR (Set the working directory): Determines the working directory within the container. It designates a specific location for application code, and all subsequent instructions (like RUN, COPY, and CMD) will be executed in this directory.

- RUN (Install dependencies): Instructs Docker to execute commands, such as using a package installer to download the specific dependencies needed for the application.
- COPY (Copy application files): Copies files from your local host machine into the designated working directory within the Docker container. Without this step, the container wouldn't have access to the application's code.
- ENV (Specify environment variables): Sets up necessary environment variables for the Docker container. Frameworks often use these variables to locate application instances to run.
- CMD (Define the default command): Specifies the default command that will be executed when the Docker container starts from the built image. This defines the program to run, instructs it to start a server, and specifies network details like host IPs and port numbers.

The .dockerignore File

A .dockerignore file works much like a .gitignore file. Because the COPY directive copies everything from the current directory, it would normally include the Dockerfile itself. Adding "Dockerfile" to the .dockerignore file ensures it is not unnecessarily copied into the container's application directory.

Key Commands for Building and Running

- Building the Image: The docker build -t <image-name>:<version> . command creates the image. The -t flag tags the image with a name and version, and the dot (.) indicates that the current directory is the build context.
- Verifying the Image: The docker image ls command checks the list of Docker images on your system to confirm creation.
- Running the Container: The docker container run command starts a container from the image.
  - The -d flag (detach) runs the container in the background.
  - The -p flag (publish) maps a port on the host machine to a port on the Docker container.
- Verifying the Container: The docker container ls command lists currently active Docker containers to ensure it is running.

Implementation:

app.py > ...

```python
from flask import Flask

app = Flask(__name__)
@app.route("/")
def hello():
    return "Hello World!"


```

Dockerfile > FROM

```dockerfile
WORKDIR /app

RUN pip install flask==2.3

COPY . /app

ENV FLASK_APP=app.py

CMD ["flask","run","--host=0.0.0.0","--port=5000"]
```

.dockerignore

```
Dockerfile
```

Administrator: C:\Windows\Sy

```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\15L\Desktop\New folder\git-demo-project>docker build . -t sample-flask-app:v1
[+] Building 8.0s (9/9) FINISHED                                                    docker:default
 => [internal] load build definition from Dockerfile                                          0.0s
 => => transferring dockerfile: 199B                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                           3.7s
 => [internal] load .dockerignore                                                             0.0s
 => => transferring context: 50B                                                              0.0s
 => [1/4] FROM docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a  1.9s
 => => resolve docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a  0.0s
 => => sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a36463b47 10.37kB / 10.37kB  0.0s
 => => sha256:06fa338add3c2ab8daeff7304a6bb204c68f67743b13553a8999605fc8097c58 1.75kB / 1.75kB  0.0s
 => => sha256:7c68b568387255c3db2b374da329795e7c25520bd232458282d62d606296e719 5.47kB / 5.47kB  0.0s
 => => sha256:2067094444255bb5bf0f8b525f10409691c57f48c48e6bebac8c3a85df4f9dd2 1.29MB / 1.29MB  1.1s
 => => sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183 14.37MB / 14.37MB  1.2s
 => => sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2 250B / 250B  0.6s
 => => extracting sha256:2067094444255bb5bf0f8b525f10409691c57f48c48e6bebac8c3a85df4f9dd2  0.1s
 => => extracting sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183  0.6s
 => => extracting sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2  0.0s
 => [internal] load build context                                                             0.4s
 => => transferring context: 13.53MB                                                          0.4s
 => [2/4] WORKDIR /app                                                                        0.1s
 => [3/4] RUN pip install flask==2.3                                                          2.0s
 => [4/4] COPY . /app                                                                         0.0s
 => exporting to image                                                                        0.1s
 => => exporting layers                                                                       0.1s
 => => writing image sha256:ad2b8021fa04bf3c090602f7d6409eed8d921693ed83223fe1f237c6938ca783  0.0s
 => => naming to docker.io/library/sample-flask-app:v1                                        0.0s

What's Next?
  1. Sign in to your Docker account → docker login
  2. View a summary of image vulnerabilities and recommendations → docker scout quickview
```

```
Administrator: C:\Windows\Sy
 => => extracting sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183                0.6s
 => => extracting sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2                0.0s
 => [internal] load build context                                                                        0.4s
 => => transferring context: 13.53MB                                                                     0.4s
 => [2/4] WORKDIR /app                                                                                   0.1s
 => [3/4] RUN pip install flask==2.3                                                                     2.0s
 => [4/4] COPY . /app                                                                                    0.0s
 => exporting to image                                                                                   0.1s
 => => exporting layers                                                                                  0.1s
 => => writing image sha256:ad2b8021fa04bf3c090602f7d6409eed8d921693ed83223fe1f237c6938ca783             0.0s
 => => naming to docker.io/library/sample-flask-app:v1                                                   0.0s

What's Next?
  1. Sign in to your Docker account → docker login
  2. View a summary of image vulnerabilities and recommendations → docker scout quickview

C:\Users\15L\Desktop\New folder\git-demo-project>docker image ls
REPOSITORY         TAG       IMAGE ID       CREATED                  SIZE
sample-flask-app   v1        ad2b8021fa04   Less than a second ago   154MB
redis              latest    c234a24851ba   13 days ago              140MB
app                latest    f881ae5ad2c9   13 days ago              161MB
myapp              latest    a5a22179b024   2 weeks ago              161MB
new_image_name     redis3    c133193b1b3e   2 weeks ago              138MB
redis              <none>    f0acdeb5ced3   2 weeks ago              140MB
node               latest    8bbb9d1bc633   24 months ago            1.1GB
t13-49             latest    acb00e2365d5   24 months ago            912MB
hello-world        latest    7e4af798f449   24 months ago            13.3kB
<none>             <none>    9c469d82cc74   24 months ago            13.3kB
ubuntu             latest    ca2b0f26964c   2 years ago              77.9MB
redis              <none>    170a1e90f843   2 years ago              138MB
redis              <none>    d1397258b209   2 years ago              138MB
hello-world        <none>    d2c94e258dcb   2 years ago              13.3kB

C:\Users\15L\Desktop\New folder\git-demo-project>
```

```
C:\Users\15L\Desktop\New folder\git-demo-project>docker container run -d -p 5000:5000 sample-flask-app:v1
ee9c7b42b5a3b9ab8665f43326b8e94bccdce54bd58698ef0356b5988fe7e0c1

C:\Users\15L\Desktop\New folder\git-demo-project>
```

```
C:\Users\15L\Desktop\New folder\git-demo-project>docker container ls
CONTAINER ID   IMAGE                 COMMAND                  CREATED                  STATUS                  PORTS
                NAMES
ee9c7b42b5a3   sample-flask-app:v1   "flask run --host=0.…"   Less than a second ago   Up Less than a second   0.0.0.0:5
000->5000/tcp   nervous_jones

C:\Users\15L\Desktop\New folder\git-demo-project>
```

Conclusion:

Hence, we have successfully studied Dockerfile instructions, built an image for a sample web application using DOCKERFILE.

⌘⌘⌘⌘⌘⌘⌘